

Introduction to Python Programming

Chapter 13: Data Structures - Intermediate

Introduction

You've already learned about lists - Python's most versatile data structure. Now it's time to explore three more powerful data structures: **tuples**, **sets**, and **dictionaries**. Each has unique characteristics that make it perfect for specific tasks. Understanding when to use each structure is key to writing efficient Python code.

1. Tuples: Immutable Sequences

A **tuple** is like a list, but it cannot be changed after creation. It's an **immutable** sequence of values.

Creating Tuples

```
# Tuple with parentheses
coordinates = (10, 20)
rgb_color = (255, 128, 0)
person = ("Alice", 25, "Calgary")

# Tuple without parentheses (also valid)
point = 5, 10
dimensions = 1920, 1080

# Single item tuple (comma required!)
single = (42,)
also_single = 42,

# Empty tuple
empty = ()
```

Key syntax note: For a single-item tuple, you MUST include a comma: `(42,)` not `(42)`

Accessing Tuple Elements

Tuples use indexing just like lists:

```
coordinates = (10, 20, 30)

print(coordinates[0])    # 10
print(coordinates[1])    # 20
print(coordinates[-1])   # 30 (last item)
```

Tuples are Immutable

Once created, you cannot modify a tuple:

```
point = (5, 10)

# This DOESN'T work
point[0] = 15 # ERROR! Tuples are immutable

# To "change" a tuple, create a new one
point = (15, 10)
```

Why Use Tuples Instead of Lists?

Use tuples when:

- Data shouldn't change (coordinates, RGB colors, dates)
- You want to protect data from accidental modification
- Slightly better performance (tuples are faster than lists)
- Using as dictionary keys (lists can't be keys, tuples can)

Use lists when:

- Data needs to change (adding/removing items)
- You need list-specific methods (append, remove, etc.)

Tuple Unpacking

You can assign tuple values to multiple variables at once:

```
coordinates = (10, 20)

# Unpack into separate variables
x, y = coordinates

print(x) # 10
print(y) # 20
```

This is incredibly useful:

```
# Swap variables
a = 5
b = 10
a, b = b, a # Now a=10, b=5

# Function returning multiple values (preview of advanced usage)
def get_dimensions():
    return 1920, 1080 # Returns a tuple

width, height = get_dimensions()
```

Iterating Through Tuples

```
person = ("Alice", 25, "Calgary")

for item in person:
    print(item)

# With index
for i in range(len(person)):
    print(f"Index {i}: {person[i]}")
```

Tuple Methods

Tuples have only two methods (because they're immutable):

```
numbers = (1, 2, 3, 2, 4, 2, 5)

# count() - how many times value appears
print(numbers.count(2)) # 3

# index() - position of first occurrence
print(numbers.index(4)) # 4
```

2. Sets: Unique Unordered Collections

A **set** is an unordered collection of unique values. No duplicates allowed!

Creating Sets

```
# Create with curly braces
numbers = {1, 2, 3, 4, 5}
fruits = {"apple", "banana", "cherry"}

# Create from a list (duplicates removed)
numbers_with_dupes = [1, 2, 2, 3, 3, 3, 4]
unique_numbers = set(numbers_with_dupes)
print(unique_numbers) # {1, 2, 3, 4}

# Empty set (must use set(), not {})
empty_set = set()
```

Important: `{}` creates an empty dictionary, not a set! Use `set()` for empty sets.

Key Characteristics of Sets

1. **No duplicates** - automatically removes duplicates
2. **Unordered** - no indexing, no guaranteed order
3. **Mutable** - can add/remove items
4. **Fast membership testing** - checking if item exists is very fast

Basic Set Operations

```
fruits = {"apple", "banana", "cherry"}

# Add an item
fruits.add("orange")
print(fruits) # {'apple', 'banana', 'cherry', 'orange'}

# Remove an item
fruits.remove("banana")
print(fruits) # {'apple', 'cherry', 'orange'}

# Check membership (very fast!)
if "apple" in fruits:
    print("Apple is in the set")

# Length
print(len(fruits)) # 3
```

Why Use Sets?

Primary uses:

1. **Remove duplicates** from a collection

2. **Fast membership testing** (is item in collection?)
3. **Mathematical set operations** (union, intersection, difference)

Removing Duplicates with Sets

```
numbers = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5]

# Convert to set, then back to list
unique_numbers = list(set(numbers))
print(unique_numbers) # [1, 2, 3, 4, 5]
```

Set Operations (Mathematical)

```
set_a = {1, 2, 3, 4, 5}
set_b = {4, 5, 6, 7, 8}

# Union - all items from both sets
print(set_a | set_b) # {1, 2, 3, 4, 5, 6, 7, 8}

# Intersection - items in both sets
print(set_a & set_b) # {4, 5}

# Difference - items in set_a but not in set_b
print(set_a - set_b) # {1, 2, 3}

# Symmetric difference - items in either set, but not both
print(set_a ^ set_b) # {1, 2, 3, 6, 7, 8}
```

Iterating Through Sets

```
fruits = {"apple", "banana", "cherry"}

for fruit in fruits:
    print(fruit)
```

Note: Order is not guaranteed! Items may appear in any order.

Sets vs Lists

Feature	List	Set
Duplicates	Allowed	Not allowed
Ordered	Yes (indexed)	No
Mutable	Yes	Yes
Speed of <code>in</code>	Slow (checks each item)	Fast (uses hashing)
Use for	Ordered collections	Unique items, fast lookup

3. Dictionaries: Key-Value Pairs

A **dictionary** stores data as key-value pairs. Think of it like a real dictionary - you look up a word (key) to find its definition (value).

Creating Dictionaries

```
# Basic dictionary
student = {
    "name": "Alice",
    "age": 20,
    "grade": "A"
}

# Various value types
mixed = {
    "name": "Bob",
    "age": 25,
    "scores": [85, 90, 92],
    "is_student": True
}

# Empty dictionary
empty_dict = {}
```

Accessing Values

```
student = {
    "name": "Alice",
    "age": 20,
    "grade": "A"
}

# Access by key
```

```

print(student["name"])    # Alice
print(student["age"])     # 20

# Using get() method (safer)
print(student.get("name"))    # Alice
print(student.get("major"))    # None (key doesn't exist)
print(student.get("major", "Undeclared")) # Undeclared (default value)

```

Why use get()?

- `dict["key"]` causes error if key doesn't exist
- `dict.get("key")` returns None if key doesn't exist (or custom default)

Adding and Modifying Items

```

student = {"name": "Alice", "age": 20}

# Add new key-value pair
student["grade"] = "A"
print(student) # {'name': 'Alice', 'age': 20, 'grade': 'A'}

# Modify existing value
student["age"] = 21
print(student) # {'name': 'Alice', 'age': 21, 'grade': 'A'}

```

Removing Items

```

student = {"name": "Alice", "age": 20, "grade": "A"}

# Remove specific key
del student["grade"]
print(student) # {'name': 'Alice', 'age': 20}

# Remove and return value
age = student.pop("age")
print(age)     # 20
print(student) # {'name': 'Alice'}

```

Dictionary Methods

```

student = {"name": "Alice", "age": 20, "grade": "A"}

# Get all keys
print(student.keys()) # dict_keys(['name', 'age', 'grade'])

```

```
# Get all values
print(student.values()) # dict_values(['Alice', 20, 'A'])

# Get all key-value pairs
print(student.items()) # dict_items([('name', 'Alice'), ('age', 20), ('grade', 'A')])

# Check if key exists
if "name" in student:
    print("Name key exists")
```

Iterating Through Dictionaries

```
student = {"name": "Alice", "age": 20, "grade": "A"}

# Iterate through keys
for key in student:
    print(key)

# Iterate through values
for value in student.values():
    print(value)

# Iterate through key-value pairs
for key, value in student.items():
    print(f"{key}: {value}")
```

Why Use Dictionaries?

Use dictionaries when:

- You need to look up values by meaningful names (not numeric indices)
- Data has natural key-value relationships (name→age, word→definition)
- You need fast lookups by key
- Representing structured data (like JSON)

Common use cases:

- Storing configuration settings
- Counting occurrences of items
- Caching/memoization
- Representing records (like database rows)

Practical Example: Word Counter


```

text = "apple banana apple cherry banana apple"
words = text.split()

word_count = {}

for word in words:
    if word in word_count:
        word_count[word] = word_count[word] + 1
    else:
        word_count[word] = 1

print(word_count)  # {'apple': 3, 'banana': 2, 'cherry': 1}

```

4. Comparing Data Structures

Quick Reference

Structure	Ordered	Mutable	Duplicates	Access Method	Primary Use
List	Yes	Yes	Yes	Index <code>[0]</code>	Ordered collections
Tuple	Yes	No	Yes	Index <code>[0]</code>	Immutable sequences
Set	No	Yes	No	Membership <code>in</code>	Unique items, fast lookup
Dictionary	Yes*	Yes	No (keys)	Key <code>["key"]</code>	Key-value pairs

*Dictionaries maintain insertion order as of Python 3.7+

When to Use Each

```

# List - ordered, changeable collection
shopping_list = ["milk", "eggs", "bread"]
shopping_list.append("butter")

# Tuple - fixed data that shouldn't change
coordinates = (10, 20)
rgb_color = (255, 128, 0)

# Set - unique items, no duplicates
unique_visitors = {"user1", "user2", "user3"}
unique_visitors.add("user1")  # No duplicate added

# Dictionary - lookup by meaningful key
user = {
    "username": "alice",
    "email": "alice@email.com",
    "age": 25
}

```

```
}  
print(user["email"])
```

5. Practical Examples

Example 1: Student Records

```
# Dictionary for structured data  
student1 = {  
    "name": "Alice",  
    "age": 20,  
    "grades": [85, 90, 92], # List within dictionary  
    "subjects": {"Math", "Science", "English"} # Set within dictionary  
}  
  
print(f"Student: {student1['name']}")  
print(f"Average grade: {sum(student1['grades']) / len(student1['grades'])}")  
print(f"Subjects: {student1['subjects']}")
```

Example 2: Find Common Elements

```
list1 = [1, 2, 3, 4, 5, 6]  
list2 = [4, 5, 6, 7, 8, 9]  
  
# Convert to sets and find intersection  
set1 = set(list1)  
set2 = set(list2)  
common = set1 & set2  
  
print(f"Common elements: {common}") # {4, 5, 6}
```

Example 3: Contact Information

```
# Dictionary of dictionaries  
contacts = {  
    "Alice": {"phone": "555-1234", "email": "alice@email.com"},  
    "Bob": {"phone": "555-5678", "email": "bob@email.com"}  
}  
  
# Access nested data  
print(contacts["Alice"]["phone"]) # 555-1234  
  
# Add new contact  
contacts["Charlie"] = {"phone": "555-9999", "email": "charlie@email.com"}
```

Example 4: Unique Word List

```
text = "the quick brown fox jumps over the lazy dog the fox"
words = text.split()

# Get unique words using set
unique_words = set(words)
print(f"Unique words: {unique_words}")
print(f"Total words: {len(words)}")
print(f"Unique words: {len(unique_words)}")
```

Example 5: Tuple for Return Values

```
def get_min_max(numbers):
    return min(numbers), max(numbers) # Returns tuple

values = [3, 7, 2, 9, 1, 5]
minimum, maximum = get_min_max(values) # Tuple unpacking

print(f"Min: {minimum}, Max: {maximum}")
```

Key Takeaways

- ✓ **Tuples** are immutable lists - use for data that shouldn't change
- ✓ **Tuple unpacking** allows assigning multiple variables at once
- ✓ **Sets** store unique values with fast membership testing
- ✓ **Sets** are unordered - no indexing or guaranteed order
- ✓ **Set operations** include union, intersection, difference
- ✓ **Dictionaries** store key-value pairs for fast lookup by key
- ✓ **Dictionary keys** must be unique and immutable (strings, numbers, tuples)
- ✓ **Use get()** for safe dictionary access without errors
- ✓ **Choose structure** based on: ordered? changeable? duplicates? access method?
- ✓ **Data structures can nest** - lists in dictionaries, sets in tuples, etc.

Reflection Questions

1. What's the main difference between tuples and lists?

2. Why do you need a comma in a single-item tuple?
 3. What happens when you add a duplicate to a set?
 4. Why are sets faster than lists for checking membership?
 5. What's the difference between `dict["key"]` and `dict.get("key")`?
 6. Can you use a list as a dictionary key? Why or why not?
 7. When would you choose a tuple over a list?
 8. What's the advantage of using a dictionary over a list?
-

Practice Exercises

1. **Coordinate System:** Create tuples for 5 points and calculate distance between two points
2. **Remove Duplicates:** Take a list with duplicates, convert to set, print unique values
3. **Student Dictionary:** Create a dictionary with student info, add/modify/delete keys
4. **Set Operations:** Create two sets of numbers, find union, intersection, and difference
5. **Word Frequency:** Count how many times each word appears in a sentence using a dictionary
6. **Tuple Unpacking:** Write a function that returns three values, unpack them into variables
7. **Contact Manager:** Build a dictionary of contacts with names as keys, phone/email as values
8. **Common Items:** Find items that appear in both lists using sets
9. **Dictionary Iteration:** Print all key-value pairs from a dictionary in a formatted way
10. **Mixed Structure:** Create a dictionary where values are lists, add/remove items from those lists